Which Language for AI, and When

Python, C++, Java and Julia: what each one is really used for, what to learn first in it, and how to choose a second language.

October 2026. Built from the TIOBE index, the Stack Overflow Developer Survey, job-posting analyses and project documentation. Sources are listed at the end.

> **Start with Python. Then pick your second language by the job you want, not by hype.**

The short answer

YOUR GOAL	START WITH	WHY
Learn AI and ML, train models, build apps on LLMs	Python	Required in most AI job postings. PyTorch, Hugging Face and the LLM tooling are Python-first.
Make models run fast and cheap: GPUs, phones, edge devices, inference engines	C++	llama.cpp, the TensorRT-LLM runtime, CUDA and self-driving stacks live here.
Add AI features to a company's existing backend or an Android app	Java (Kotlin on Android)	Spring AI, LangChain4j, ONNX Runtime for Java; fits the systems enterprises already run.
Simulation, modeling and numerical research, or replacing MATLAB	Julia	Near-C speed for numeric code and a strong scientific ML ecosystem (SciML). Niche elsewhere.

The four at a glance

LANGUAGE	TIOBE, SEPT 2026	LEARNING CURVE	SPEED	WHERE IT SHOWS UP IN AI
Python	#1 (17.76%)	Gentle	Slow itself; fast through C/C++/CUDA libraries	Research, training, data work, LLM apps
C++	#3 (8.67%)	Steep	Fastest of the four	Inference engines, GPU code, robotics, self-driving
Java	#4 (7.54%)	Moderate	Fast on the JVM	Enterprise systems, Spark, Android
Julia	#21 (0.74%)	Gentle to moderate	Close to C on numeric code	Scientific computing, simulation, SciML

TIOBE tracks search-based popularity and is not a direct measure of how much a language is used. The learning-curve and speed columns are a summary of common experience, not benchmarks.

How to use this guide

Each language gets one page: where it is used in AI, what to learn in order, a small example, and when to skip it. Then one page on choosing a second language and what the short version of this topic leaves out, followed by sources.

LANGUAGE 1 OF 4

Python

Use it for: learning AI, training models, data work, building apps on LLMs.

Why it comes first

- **Developers:** 58.1% of Stack Overflow's 2026 respondents used Python, and it was the most common language used alongside AI tools (39%, with JavaScript at 38%).
- **Jobs:** Python appears in 71% of 903 AI engineer postings on Glassdoor US (365 Data Science), in 77% of a 31-posting sample (Heavybit) and in 61.5% of 43,480 US postings (Axial). The numbers differ by method; the ranking does not.
- **Research:** JetBrains' May 2026 comparison says about 85% of deep learning papers at top conferences use PyTorch. Other estimates run lower, down to about 55%, but all put PyTorch first.
- **Index:** TIOBE still has Python first at 17.76% in September 2026, well ahead of C at 10.28%, though its rating has slipped through 2026 (18.53% in August, 7.61 points below a year earlier).

What to learn, in order

STEP	WHAT
1. Core Python	Functions, classes, virtual environments, type hints, reading tracebacks.
2. The data stack	NumPy, pandas, matplotlib.
3. Classical ML	scikit-learn: train/test splits, metrics, overfitting.
4. Deep learning	PyTorch, then Hugging Face Transformers.
5. LLM applications	Provider APIs, retrieval (RAG), agents, and evaluating outputs.

In the Heavybit sample of AI engineer postings, evaluations were mentioned in 61%, LangChain or LangGraph in 32% and MCP in 16%. It is a small sample, so treat it as a direction rather than a measurement.

Starter example: a tiny PyTorch classifier

```
import torch, torch.nn as nn

model = nn.Sequential(nn.Linear(2, 8), nn.ReLU(), nn.Linear(8, 1))
opt = torch.optim.Adam(model.parameters(), lr=0.01)

x = torch.randn(64, 2)
y = (x.sum(dim=1, keepdim=True) > 0).float()   # label: is the sum positive?

for step in range(200):
    loss = nn.functional.binary_cross_entropy_with_logits(model(x), y)
    opt.zero_grad()
    loss.backward()
    opt.step()

print(float(loss))
```

Watch out for

- **Python is not fast on its own.** The speed comes from C, C++ and CUDA code inside NumPy, PyTorch and similar libraries.
- **The GIL is now optional.** Python 3.14 officially supports a free-threaded build without the Global Interpreter Lock. It is opt-in, single-threaded code runs roughly 5 to 10% slower, and not every library supports it yet.
- **Applied AI roles dominate postings.** The Heavybit sample found employers asking for people who build with LLMs more than people who train models, and only 3% of those postings targeted new graduates.

LANGUAGE 2 OF 4

C++

Use it for: making models run fast on GPUs, phones and edge devices; AI infrastructure; robotics and self-driving.

Where C++ shows up

AREA	EXAMPLE	WHAT IT DOES
Local LLM inference	llama.cpp	A C++ inference engine tuned for CPUs, Apple Silicon, consumer GPUs, phones and Raspberry Pi. Ollama is a wrapper over it.
NVIDIA serving	TensorRT-LLM	NVIDIA's toolkit for building your own LLM inference engine, with a Python API and C++ and Python runtimes.
Python-free deployment	LibTorch	Load a TorchScript model and run inference from C++ with no Python at runtime.
Self-driving	Apollo, Autoware	Open-source driving stacks that handle live sensor data and heavy perception and planning under real-time limits.
GPU kernels	CUDA C++	Kernel optimization, memory hierarchy and profiling on NVIDIA hardware.

What the jobs look like

– NVIDIA's CUDA driver roles ask for strong C and C++. 2026 listings showed US salary ranges of $152,000 to $287,500 (senior) and $272,000 to $431,250 (principal), plus equity.
– Its entry-level AI developer-technology role asks for C/C++ and prefers CUDA C/C++ and GPU architecture knowledge.
– Most AI engineer postings do not require C++. In the Heavybit sample, C++, Java and Go were mentioned, likely for other parts of the codebase. These are specialist roles with a higher bar.

Measure before you rewrite

PyTorch forum guidance is that when your tensors are reasonably large, Python overhead is rarely the bottleneck. One GitHub user reported a LibTorch model running at about 24 ms versus 18 ms in Python on their setup. Move to C++ for deployment reasons (no Python at runtime, embedded targets, memory control, latency), not because you expect an automatic speedup.

Learn in this order

– Modern C++ (17 or 20): RAII, smart pointers, the STL, templates at a working level.
– CMake and a debugger/profiler; then threads and atomics.
– CUDA basics: kernels, memory hierarchy, occupancy.
– Project: export a PyTorch model to TorchScript and run it from C++ with LibTorch, then read how llama.cpp loads and runs a model.

Starter example: running a TorchScript model from C++

```
#include <torch/script.h>
#include <iostream>

int main() {
    auto model = torch::jit::load("model.pt");          // exported from Python
    auto input = torch::randn({1, 3, 224, 224});
    auto output = model.forward({input}).toTensor();
    std::cout << output.sizes() << std::endl;
}
```

Skip it for now if

You are still learning how models train and evaluate. C++ first slows that down; learn the ideas in Python, then take C++ on when you have a performance or deployment problem to solve.

LANGUAGE 3 OF 4

Java

Use it for: putting AI into existing enterprise systems and, with Kotlin, Android apps.

Where Java fits

- **LLM features in Spring backends:** Spring AI and LangChain4j provide chat, embedding and retrieval building blocks.
- **Serving trained models:** ONNX Runtime for Java loads a model at startup inside a Spring Boot service. Deeplearning4j and Oracle's Tribuo cover training and classical ML on the JVM.
- **Big data:** Apache Spark and its MLlib sit in the Java ecosystem and run many enterprise ML pipelines.
- **Jobs:** Java shows up in 22% of the 903 AI engineer postings in the 365 Data Science analysis (13% in the Heavybit sample), most often as the language of the system around the model.

The Android part, precisely

Google's Agent Development Kit for Kotlin reached 1.0 with Android extensions: on-device models through LiteRT-LM (generally available) and Gemini Nano through ML Kit (beta at launch). The Android documentation says the library supports Kotlin and Java, but the newest API is Kotlin-first with Java interoperability. So Java works for mobile AI, and Kotlin is where the newest tools arrive. If Android is your target, learn Kotlin alongside Java.

Learn in this order

- Java 21 basics, collections, streams, records.
- Spring Boot: REST endpoints, dependency injection, configuration.
- Spring AI or LangChain4j for LLM calls and retrieval.
- ONNX Runtime for Java: export a model from Python, serve it from a service.
- Kotlin and the Android AI stack if mobile is the goal.

Starter example: a chat endpoint with Spring AI

```
@RestController
class ChatController {
    private final ChatClient chat;

    ChatController(ChatClient.Builder builder) {
        this.chat = builder.build();
    }

    @GetMapping("/ask")
    String ask(@RequestParam String q) {
        return chat.prompt().user(q).call().content();
    }
}
```

Skip it for now if

Your goal is training models or following new research. Java's deep learning frameworks trail Python's in access to new techniques, and most papers and pretrained models arrive in PyTorch first.

LANGUAGE 4 OF 4

Julia

Use it for: scientific computing, simulation, and ML that mixes physical models with data.

Where Julia fits

- **Popularity:** TIOBE had Julia at 21st in September 2026 with 0.74%, just behind COBOL and close to returning to the top 20. TIOBE's CEO says it has taken ground from MATLAB in numerical work and modeling, but it is not yet a general-purpose language.
- **Speed with an interactive feel:** Julia compiles code just in time, which gives C and Fortran-like speed on numeric code while staying usable from a REPL or notebook.
- **Scientific ML:** Flux.jl is a pure-Julia ML library. The SciML project (differential equation solvers plus ML) is used by the Climate Modeling Alliance, the New York Fed, Pumas-AI (pharmaceutical modeling) and Brazil's space research institute INPE.
- **Works with Python:** PythonCall lets Julia and Python call each other, and DifferentialEquations.jl can be used from Python via diffeqpy.
- **Current version:** Julia 1.13.0 is out, and 1.12 is now officially unmaintained.

The honest downsides

- **Compile latency.** The first run of a function pays a compile cost (the "time to first plot" complaint). The 2025 Julia user survey still lists slow compile and first-plot time among the least-liked technical points, along with difficulty producing self-contained binaries.
- **Small community and few jobs.** The same survey names "not enough Julia users" as the biggest non-technical problem.
- **Not for hard real-time work.** Memory allocation behavior makes it a poor fit for strict real-time systems.
- **Smaller ecosystem.** Some packages are thinly maintained, and deployment into a C++, Java or Python company stack takes effort.

Learn in this order

- Install with juliaup; learn the REPL, the package manager, arrays and broadcasting.
- Multiple dispatch and types, the idea that makes Julia code fast and composable.
- Flux.jl for neural networks; DifferentialEquations.jl and the SciML packages for dynamics.
- Project: reproduce a model from your field in Julia and compare it with your current MATLAB or Python version.

Starter example: a tiny Flux model

```
using Flux

model = Chain(Dense(2 => 8, relu), Dense(8 => 1))
x = randn(Float32, 2, 64)
y = Float32.(sum(x, dims=1) .> 0)             # label: is the sum positive?

opt_state = Flux.setup(Adam(0.01), model)
for step in 1:200
    grads = Flux.gradient(m -> Flux.logitbinarycrossentropy(m(x), y), model)
    Flux.update!(opt_state, model, grads[1])
end
```

Skip it for now if

You want the widest job market or are still learning ML basics. Julia is strongest when you already work in simulation, modeling or numerical research.

DECIDING

Choosing your second language

Python first, then follow the job you want.

IF YOU WANT TO...	PICK	FIRST PROJECT
Ship AI features inside a company's backend or an Android app	Java (and Kotlin for Android)	A Spring Boot service that answers questions over your own documents.
Make models cheaper and faster to run, or work close to GPUs and devices	C++, then CUDA	Run a TorchScript model from C++, then profile it.
Work on simulations, models of physical systems, or replace MATLAB	Julia	Port one model from your field and compare speed and clarity.
Build AI products as quickly as possible	Stay in Python; add SQL and TypeScript	A retrieval app with an evaluation set, deployed with a simple web front end.

In one analysis of AI engineer postings, the languages employers screened for most were Python, TypeScript, Java, JavaScript and Go. SQL was used by 58.4% of developers in the 2026 Stack Overflow survey.

A 12-week start in Python

WEEKS	FOCUS	OUTPUT
1 to 3	Core Python and the command line	Three small scripts that read files and call an API.
4 to 6	NumPy, pandas, scikit-learn	A cleaned dataset and a baseline model with honest metrics.
7 to 9	PyTorch and Hugging Face	A fine-tuned or trained small model, with the training loop written by you.
10 to 12	An LLM application	A retrieval app with an evaluation set, published on GitHub with a readable README.

This plan is a suggestion built from the skills the job-posting analyses list; adjust the pace to your time.

What the short version leaves out

- **Python's speed comes from other languages.** Heavy computation runs in compiled C, C++ and CUDA code underneath. That is why C++ matters even if you never write it.
- **"Java for mobile AI" is half the story.** It works, but Android's newest AI tooling is Kotlin-first.
- **Popularity indexes are not usage.** TIOBE counts search activity. Python's rating has dropped in 2026 and it is still far ahead.
- **Julia is a specialist's tool.** It is growing in scientific work and is still a small fraction of the market.
- **Pay figures are mostly US.** The median for US AI engineer postings in one 2026 analysis was about $190,000 (typical range $160,000 to $225,000, 3,828 postings). Pay in India differs, so check local listings.

Sources

Figures were checked against these pages in October 2026. Job-posting percentages come from analyses with different sample sizes and methods; the sample size is stated where it matters.

1. TIOBE Index for September 2026: Python Drops Below 18% as Julia Nears Top 20 (TechRepublic)
   https://www.techrepublic.com/article/news-tiobe-september-2026-julia-nears-top-20/
2. Julia nearly back in the top 20 most popular programming languages (Techzine)
   https://www.techzine.eu/news/devops/144127/julia-nearly-back-in-the-top-20-most-popular-programming-languages/
3. TIOBE Index for August 2026: Java Nears C++ as Rust Holds No. 10 (TechRepublic)
   https://techrepublic.com/article/news-tiobe-august-2026-java-nears-c-plus-plus/
4. The results of the 2026 developer survey are here (Stack Overflow blog)
   https://stackoverflow.blog/2026/10/06/the-results-of-the-2026-developer-survey-are-here/
5. Stack Overflow survey shows developers trust AI with caveats (IT Brief)
   https://itbrief.co.uk/story/stack-overflow-survey-shows-developers-trust-ai-with-caveats
6. PyTorch vs. TensorFlow: Choosing the Right Framework in 2026 (JetBrains blog)
   https://blog.jetbrains.com/pycharm/2026/05/pytorch-vs-tensorflow-choosing-framework-2026/
7. AI Engineer Job Outlook 2026, 1,000 job postings (365 Data Science)
   https://365datascience.com/career-advice/career-guides/ai-engineer-job-outlook-2025/
8. So You Want to Be an AI Engineer? (Heavybit)
   https://heavybit.com/library/article/so-you-want-to-be-an-ai-engineer
9. AI Engineering Jobs in 2026: A Data-Backed Market Map (Axial)
   https://axialsearch.com/insights/ai-ml-engineering-jobs/
10. How to Hire an AI Engineer (2026) (Recruiting from Scratch)
    https://www.recruitingfromscratch.com/blog/how-to-hire-a-ai-engineer
11. Goodbye GIL: Exploring Free Threaded Python 3.14 (Adarsh Divakaran)
    https://adarshd.substack.com/p/goodbye-gil-exploring-free-threaded
12. Python 3.14 No-GIL explained and performance analysis
    https://engineersmeetai.substack.com/p/python-314s-no-gil-explained-and
13. Inference engines: llama.cpp, vLLM, TensorRT-LLM (Nanonets)
    https://nanonets.com/cookbooks/structured-llm-outputs/constrained-decoding/inference-engines
14. LLM-Inference-Bench: Inference Benchmarking of LLMs on AI Accelerators (arXiv)
    https://arxiv.org/pdf/2411.00136
15. Serving a PyTorch Model in C++ Using LibTorch
    https://naveenmarthala.substack.com/p/serving-pytorch-model-in-cpp-with
16. Custom C++ extensions vs LibTorch (PyTorch forums)
    https://discuss.pytorch.org/t/custom-c-extensions-vs-libtorch/133909
17. Why does the C++ LibTorch performance seem slower than PyTorch? (GitHub issue)
    https://github.com/prabhuomkar/pytorch-cpp/issues/127
18. Open-Source Autonomous Driving Software Platforms: Comparison of Autoware and Apollo (arXiv)
    https://arxiv.org/pdf/2501.18942
19. OpenMP API Speeds Up Autonomous Driving Codes
    https://www.openmp.org/blog/openmp-api-speeds-up-autonomous-driving-codes/
20. Senior Software Engineer, CUDA UMD - GPU Kernel Scheduling (NVIDIA listing)
    https://jobs.anitab.org/companies/nvidia/jobs/91918557-senior-software-engineer-cuda-umd-gpu-kernel-scheduling
21. Principal Engineer, CUDA UMD - GPU Kernel Scheduling (NVIDIA listing)
    https://jobs.anitab.org/companies/nvidia/jobs/69576053-principal-engineer-cuda-umd-gpu-kernel-scheduling
22. Developer Technology Engineer, AI - New College Grad 2026 (NVIDIA listing)
    https://hitmarker.net/jobs/nvidia-developer-technology-engineer-ai-new-college-grad-2026-1586940
23. Java and AI: A Practical Guide for Production Systems (ThirstySprout)
    https://thirstysprout.com/post/java-and-ai
24. Eclipse Deeplearning4j project page
    https://projects.eclipse.org/node/12405
25. Build ADK agents for Android (Android Developers)
    https://developer.android.com/ai/adk
26. Google Releases ADK for Kotlin 1.0 with On-Device Android Agent Support
    https://techjacksolutions.com/ai-brief/google-adk-kotlin-1-0-on-device-android-agents/

27. ADK for Kotlin v1.0.0 release notes (GitHub)
https://github.com/google/adk-kotlin/releases/tag/v1.0.0

28. Flux: The Julia Machine Learning Library
https://fluxml.ai/Flux.jl/previews/PR2112

29. SciML project page (NumFOCUS)
https://numfocus.org/project/sciml

30. The State of Julia for Scientific Machine Learning (arXiv)
https://www.arxiv.org/pdf/2410.10908

31. 2025 Julia User and Developer Survey
https://julialang.org/assets/2025-julia-user-developer-survey.pdf

32. Julia 1.6 and the time-to-first-plot problem (LWN)
https://lwn.net/Articles/856819

33. Julia v1.13.0 has been released (Julia Discourse)
https://discourse.julialang.org/t/julia-v1-13-0-has-been-released/139326

34. This month in Julia world, October 2025 (PythonCall improvements)
https://julialang.org/blog/2025/10/this-month-in-julia-world/

35. Introduction to Julia, Cambridge course slides
https://www.cl.cam.ac.uk/teaching/current/TeX+Julia/julia-slides.pdf